

PARALLAX

Gunshot Detection & Localization System

A distributed, multi-sensor system that listens for gunfire, decides whether a sound is actually a gunshot, and works out where it came from — direction, and where possible, exact range.

STATUS: SOFTWARE PROTOTYPE. No hardware has been built. No real gunshots were recorded. Every waveform in this project is a physics-based simulation, and every accuracy number quoted in this document comes from testing against that simulation, not from a field trial. This is stated plainly here because a simulated result presented as a real-world measurement is the fastest way to lose credibility with anyone technical reviewing this project.

1. The Problem We're Solving

This project is built around a real problem statement (from Smart India Hackathon): **build a system that can detect a gunshot and tell you which direction it came from, using an array of microphones.**

The idea behind it is simple and important. When a shot is fired at a soldier or a patrol, the sound reaches human ears in under a second — but a human being startled by a gunshot is not a reliable direction-finder. People freeze, misjudge direction, or simply can't react fast enough. A sensor-based system doesn't get startled. If several microphones can hear the same sound arrive at very slightly different times, geometry alone can work out where the sound came from — no guessing required.

But direction alone is only half the answer a commander actually needs. Two questions matter in the moment right after a shot is fired:

A basic microphone array can answer the first question well. Answering the second question — *distance* — turns out to be a much harder problem, and solving it properly, without cheating, is the core technical achievement of this project.

2. What Makes This Hard (and What Makes It Interesting)

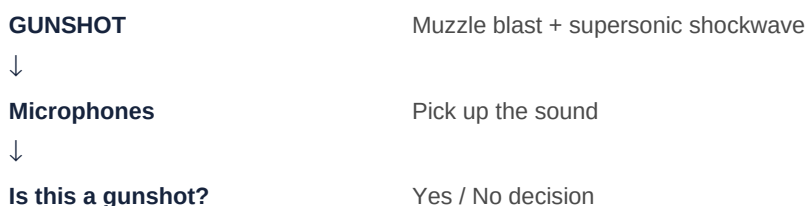
Most simple approaches try to guess distance from how *loud* the gunshot sounds. This does not work reliably: the loudness a microphone picks up depends on the weapon, the ammunition, which way the barrel was pointing, the terrain, and the weather. Two shots fired from the exact same spot with two different weapons can sound completely different in loudness by the time they reach a listener 300 metres away. Any system that estimates range from loudness alone is making an educated guess and presenting it as a measurement — and this project deliberately refuses to do that.

Instead, this project uses a genuinely clever piece of physics: **most rifle bullets travel faster than the speed of sound.** A bullet moving faster than sound drags a small sonic boom behind it — the same physics behind the “crack” you hear from a passing supersonic jet. This shockwave is a completely separate sound event from the gun's muzzle blast (the “bang”), and it arrives at a microphone *before* the bang does, and from a slightly different direction.

That tiny difference — in timing, and in direction — turns out to contain everything needed to work out how far away the shooter is, **without needing to know what weapon was used.** This is the heart of the project, and is explained properly in Section 5.

3. The Big Picture: How a Shot Gets Detected and Located

Here is the full journey, start to finish, in plain terms:



↓	NO → stop here, nothing else happens
Which direction?	Compass bearing recovered from mic timing
↓	
Supersonic + close enough for crack + bang?	Decides whether range can be solved
↓	YES → direction + distance NO → direction only
Combine all teams	If more than one team heard it, fuse into one best answer
↓	
FINAL RESULT	Direction, distance, map position, confidence score

Every stage of this pipeline exists in working, tested code (not just a design document) and is explained in its own section below.

4. Section: Is It Actually a Gunshot? (Classification)

Before any direction or distance math runs, the system has to answer a simpler but more important question: *is this sound even a gunshot?* A firecracker, a door slamming, a car backfiring, and a real gunshot can all sound similar to a simple energy detector — sharp, sudden, loud.

The system measures about two dozen properties of the sound it picked up: how quickly it rises to full volume, how it fades away, what frequencies it contains, and (importantly) some properties specific to the shockwave shape described above, since a shockwave has a distinctive “sharp spike then mirror-image dip” shape that a firecracker's blast doesn't share.

These properties are fed into a trained model — not a hand-written rule, but a decision system trained on many examples — which outputs a probability: “92% likely this is a gunshot.” If that number is too low, the system stops right there. No direction, no distance, nothing shown on a screen. This matters a lot in practice: a system that lights up a commander's map for every door slam and firecracker within earshot quickly gets ignored, and an ignored warning system is a useless one.

Tested result (on simulated data): correctly identifies about 98% of real gunshots, while rejecting firecrackers, engines, and footsteps with a very low false-alarm rate.

5. Section: Finding the Direction (and, When Possible, the Distance)

This is the technical core of the whole project, so it's worth explaining carefully and patiently.

5a. Direction is the “easy” part

If you have several microphones spread a small distance apart, sound reaches each one at a very slightly different moment — the same reason your two ears can tell if someone is speaking from your left or right. By measuring those tiny timing differences across all the microphone pairs, some straightforward geometry (no guessing, no machine learning — just solving a system of equations) gives a compass bearing to where the sound came from, typically accurate to under a degree.

5b. Distance is the genuinely hard part

As explained in Section 2, most supersonic rifle rounds produce **two** separate sounds: the shockwave (“crack”) and the muzzle blast (“bang”). A microphone close enough to the bullet's path hears both. Here's the key insight, explained step by step:

Put together, a **single** microphone team that hears both sounds clearly can work out the shooter's direction, distance, and the bullet's speed — all at once, and without ever needing to know what weapon fired the shot. That last property is what makes this approach genuinely useful in the field: a system that has to recognise a weapon's specific sound signature to work is fragile and easy to fool; this system works from physics alone.

When this doesn't work: if the shot came from too far away, or the bullet was subsonic (some pistol rounds, for instance), the crack either never reaches the microphone or can't be told apart from the bang. In that case, the system is honest about it: it reports the direction only, and leaves distance blank rather than guessing. A wrong number is worse than an honest “I don't know,” and this principle is applied everywhere in the project, not just here.

6. Section: Multiple Teams Working Together (Fusion)

A single listening post is useful, but several listening posts working together are far more powerful — and far more trustworthy. This project's “fusion” layer is what combines everyone's information into one final answer, and it follows one hard rule throughout: **never silently trust one source of information over another. Cross-check everything, and say so plainly when two measurements disagree.**

Concretely, when a shot is heard by more than one location, up to three independent ways of estimating distance can exist at once:

If two or more of these methods land on similar answers, the system combines them — giving more weight to whichever measurement is more precise — into one tighter, more confident final answer. If the methods land on *noticeably different* answers, the system does **not** quietly average them together and hope for the best. It flags the disagreement openly, and falls back to whichever single measurement it trusts most, rather than presenting a confident-looking number that's actually a blend of a good measurement and a bad one.

This same “don't silently trust one thing” philosophy also shows up one level down: when the direction-finding math is looking at 15 different pairs of microphones and one of those 15 measurements looks like an outlier (likely caused by an echo off a nearby wall), the system detects and drops that one bad measurement before it can drag the final answer off course.

7. Section: The Final Output

Once everything above has run, what actually gets shown is deliberately simple and clean — five pieces of information, no more:

```
{
  "direction": "N42.0E",
  "range_m": 300.1,
  "latitude": 28.616949,
  "longitude": 77.211075,
  "accuracy_pct": 87.3
}
```

Field	What it means
direction	Compass bearing, written the way a map-reader would say it out loud (“42° east of north”), not a bare number.
range_m	Distance to the shooter in metres — or blank if the shot was out of range for the crack/bang method and no tear
latitude / longitude	The shooter's estimated position, ready to drop onto a map.
accuracy_pct	An honest confidence score, from testing how much the estimate would wobble under slightly noisier measurem

This is intentionally the *entire* output. A commander in the field, or a piece of mapping software receiving this over a network, does not need — and should not have to wade through — internal detail like which specific mathematical method produced the answer. That detail still exists internally for engineers and for debugging; it's simply not part of what gets sent out.

8. Section: Built to Fail Honestly, Not Silently

A theme runs through every part of this project, worth calling out on its own: **the system is designed to say “I don't know” rather than invent an answer.** A few concrete examples, all enforced in the actual code, not just described in a design document:

This mindset — under-promise, and be transparent about uncertainty — is what separates a system a commander can actually trust from one that just produces impressive-looking numbers.

9. How Thoroughly This Has Been Tested

Everything described above is backed by an automated test suite (60 tests at time of writing) and by large-scale randomised stress-testing, not just a handful of hand-picked demo examples. A few concrete results from that testing:

10. What's Still a Work in Progress

In the interest of the same honesty this project tries to build in everywhere else: a few things are deliberately not finished yet.

11. Why This Matters — The Practical Case

Step back from the algorithms for a moment and consider the actual situation this is built for: a soldier or a security team is under fire, and the single most valuable piece of information in that moment is “**where is the danger, and how far away is it?**” Every second spent figuring that out by ear, or by guesswork, is a second of exposure.

What this project demonstrates is that a genuinely useful answer to that question does not require expensive, exotic hardware or classified military-grade sensors. It requires:

The specific technique at the heart of this project — reading a bullet's speed and the shooter's range directly from the geometry of its own shockwave, rather than from a database of “what different guns sound like” — is what makes the system **weapon-agnostic**. It doesn't need to be told, trained on, or updated for every new rifle or ammunition type that might be used against it, which is precisely the kind of assumption a real adversary would try to exploit against a less careful system.

Beyond the immediate military and law-enforcement use case this was designed for, the same underlying idea — multiple simple sensors, physics-based cross-checking, and honest confidence reporting instead of guesswork — has real value anywhere sound needs to be traced back to its source under uncertainty: campus and public-safety shot-detection systems, industrial safety monitoring for equipment failures, wildlife and poaching-detection acoustic networks, and any other setting where “where did that sound come from, and can we trust this estimate” is a question worth answering carefully rather than quickly.

PARALLAX — software prototype. No hardware built, no field recordings made. Generated from the project's own README and source.